Prometheus — ESP32C3Mini

Monitoramento UDP

Métrica	Tipo	Escopo
`esp_udp_listener_up`	gauge	1 quando bind da porta UDP 4210 foi bem-sucedido, 0 se falhou.
`esp_udp_packets_received_total`	counter	Datagramas não vazios reportados por parsePacket, incluindo comandos inválidos.
`esp_udp_replies_sent_total`	counter	Respostas completamente escritas e aceitas por endPacket.
`esp_udp_send_errors_total`	counter	Uma falha por tentativa de resposta com erro em beginPacket, escrita ou endPacket.

Todos incluem o label global `device` (MAC). Contadores são desde boot e não são zerados por `reset_counters` (que atua somente nos persistentes de boot/WDT). Listener indica resultado local do bind, não teste de conectividade externa. Envio UDP bem-sucedido não comprova entrega ao destinatário. Comandos que reiniciam antes de retornar não chegam ao fechamento/contabilização da resposta. Pacotes somente com espaços são recebidos, mas não geram resposta; datagramas vazios que parsePacket não reporta não são contados. Mensagens maiores que 255 bytes ou incompletas geram erro textual sem executar comando truncado.

MQTT do Home Assistant

`mqtt_connected` é gauge 1 quando o cliente MQTT está conectado ao broker, 0 caso contrário, inclusive quando desativado. `mqtt_messages_sent`, `mqtt_messages_received` e `mqtt_publish_errors` agora são **counters reais** desde boot, em vez de placeholders NaN. Contam chamadas publish bem-sucedidas, mensagens entregues ao callback e chamadas publish que falharam, respectivamente. Não contam ACKs/pings, nem garantem consumo remoto. Incluem descoberta, disponibilidade e limpeza de tópicos. `esp_metric_supported{group="mqtt"}` vale 1. Veja homeassistant.md para instalação e configuração.

Painéis de SD sem dados

Por solicitação do usuário, o Mini não exporta nenhuma série `sdcard_*`, incluindo capacidade, espaço livre/usado, percentual, logs e erros de SD. Não publica zero nem NaN para esse grupo: os painéis recebem ausência de séries para representar **No data**. Permanece apenas `esp_metric_supported{group="sdcard"} 0`, indicador de suporte do firmware.

Após gravar o firmware, aguarde a próxima coleta. Painéis com consultas históricas ainda podem encontrar amostras antigas no intervalo selecionado. Se houver substituição de valores ausentes por zero ou texto personalizado no Grafana, essa configuração do painel precisa permitir **No data**. As referências a métricas SD nas tabelas abaixo são catálogo de nomes, não séries atualmente exportadas; a regra desta seção prevalece para SD.

Idade dos dados externos

`esp_weather_data_age_seconds` é um gauge em segundos desde a última consulta bem-sucedida de Porto Alegre, com o label global `device` (MAC). Segue o nome usado no M5NanoC6. Exemplo para o painel com variável de modelo:

```
esp_weather_data_age_seconds
and on(instance, job) esp_info{model="$device"}
```

O comando `clima_age` (alias `age`), disponível por serial, UDP e comando web, mostra segundos e minutos. O timer é monotônico de 64 bits e independe do NTP. A idade não é persistida: antes do primeiro sucesso de cada boot, a métrica retorna NaN e o comando informa indisponibilidade. Uma tentativa que falha não zera o relógio. Mede idade do cache recebido pela placa, não o horário de observação/modelagem meteorológica da Open-Meteo.

Consultas de Porto Alegre no Grafana

As leituras externas incluem `source="open_meteo"`, `location="porto_alegre"` e o label global `device` (MAC). `esp_info` mantém `model="ESP32C3Mini"`. Use as consultas solicitadas:

```
sensor_temperature_celsius{source="open_meteo",location="porto_alegre"}
and on(instance, job) esp_info{model="$device"}
```

```
sensor_humidity_percent{location="porto_alegre"}
and on(instance, job) esp_info{model="$device"}
```

Nessas duas consultas, `$device` precisa conter **ESP32C3Mini**. Configure a variável por `label_values(esp_info, model)`. Se a variável existente contém MACs (`label_values(esp_info, device)`), mantenha-a e use `esp_info{device="$device"}` no lado direito em vez de filtrar por `model`. Os labels `instance` e `job` são acrescentados pelo Prometheus na coleta. Após enviar o firmware, aguarde uma nova coleta para surgirem os novos labels.

Referência: **19/10/2026**. Implementação: ServicoPrometheus.cpp. Veja também comandos.md e changelog.md.

Configuração web e coleta

O Prometheus **coleta** por HTTP. Não há envio ativo, Pushgateway ou alteração remota da configuração do servidor.

Serviço	Porta padrão	Rota
Dashboard e configuração	80, fixa	`/`, `/config`, `/prometheus`
Métricas na placa	9120, configurável	`/metrics`

Serviço	Porta padrão	Rota
Intervalo de coleta	15 segundos, configurável	YAML gerado

Abra `http://IP_DA_PLACA/prometheus` pelo link no dashboard. Preencha o IP/nome do servidor Prometheus, a porta de métricas da placa e o intervalo de coleta. O endereço do servidor é apenas uma referência administrativa no YAML gerado, não destino de envio ou lista de permissões. Aceita IPv4 ou DNS sem protocolo, caminho, porta ou literal IPv6.

Portas válidas: 1–65535, exceto 80 e 3232 (OTA). O intervalo aceita 5–3600 segundos. Dados inválidos retornam HTTP 400. A configuração é salva em um único registro versionado nas Preferences: namespace `prometheus`, chave `config`. A gravação é conferida por leitura. Falha retorna HTTP 500 e mantém o estado ativo. Troca de porta aplicada no próximo ciclo principal, sem reboot e sem mudar a porta 80. Não há gravações NVS por coleta.

Copie o trecho mostrado na página ou abra `/prometheus/config.yml` e acrescente o job ao `scrape_configs` existente no servidor, sem duplicar essa chave YAML. Recarregue o Prometheus. O intervalo no YAML acompanha a configuração salva (padrão 15 segundos):

```
scrape_configs:
  - job_name: 'esp32c3mini'
    scrape_interval: 15s
    metrics_path: /metrics
    static_configs:
      - targets: ['192.168.0.122:9120']
```

`192.168.0.122` é o IP do ESP32C3Mini, não um endereço ilustrativo. Uma captura anterior mostrou esse target como `DOWN` com `connection refused`; isso indica que, naquele momento, nada aceitava conexões na porta 9120. Se voltar a ocorrer, confirme que o firmware com o exporter está gravado, que a porta configurada na página `/prometheus` é 9120 e teste `http://192.168.0.122:9120/metrics` na rede local. Quando o target aparece `UP`, o Prometheus já está coletando; isso não cria automaticamente datasource ou painel no Grafana.

O endereço preenchido identifica o servidor Prometheus apenas para referência; a porta padrão do servidor (normalmente 9090) não é uma configuração da placa. Reserve o IP no roteador. Trocar a porta na placa exige atualizar também o target no servidor. O intervalo configurado não altera o DHT22 (mínimo de dois segundos entre tentativas) nem o clima externo (15 minutos).

`/metrics` retorna `text/plain; version=0.0.4; charset=utf-8`, `HELP`, `TYPE` e amostras numéricas. Labels escapam aspas, barras e quebras de linha. Todas as séries incluem `device` com o MAC Wi-Fi da placa, igual ao identificador usado pelo ESP32-C3 Gateway; valores indisponíveis são **NaN**, nunca valores inventados.

Visualização no Grafana

No Grafana, adicione uma fonte de dados do tipo **Prometheus** apontando para o servidor Prometheus (normalmente `http://localhost:9090` se ambos rodam na mesma máquina). Não use `http://192.168.0.122:9120` como URL da fonte: esse endereço é o target de métricas que o Prometheus

coleta. Se o Grafana roda em container, use um endereço acessível a partir desse container para chegar ao Prometheus; nesse caso `localhost` aponta para o próprio container do Grafana.

Salve e teste a fonte de dados. Em **Explore**, selecione essa fonte e execute uma consulta PromQL por vez, por exemplo:

- `up{instance="192.168.0.122:9120"}` verifica se o Prometheus coleta o target.
- `esp_uptime_seconds{device="$device"}` consulta o device selecionado.
- `sensor_local_temperature_celsius{device="$device"}` consulta a temperatura local.

A variável de dashboard pode usar a consulta `label_values(esp_info, device)` e os painéis podem filtrar as séries com `{device="$device"}`. O valor `device` é o MAC Wi-Fi da placa, estável entre conexões STA e AP. O rótulo `instance` é adicionado pelo Prometheus e identifica o target. Se a configuração usa outro IP ou porta, ajuste a consulta de `up` conforme os rótulos exibidos no Prometheus.

Se o Mini já está `UP` no Prometheus mas não aparece ao selecionar um device no dashboard, atualize e grave este firmware: as séries anteriores do Mini não tinham o label `device` em todas as métricas. Depois do scrape novo, o MAC do Mini passa a aparecer na variável e os painéis existentes que filtram por `device` podem consultá-lo. O histórico antigo sem esse label não é alterado.

Identificação e disponibilidade

Métrica	Tipo e significado
`esp_info`	Gauge 1, labels `device` (MAC Wi-Fi), `chip`, `model`.
`esp_firmware_version`	Gauge 1, labels `version`, `build`, `sha256` do firmware.
`esp_ip_address`	Gauge 1, label `ip` STA ou AP atual.
`esp_mac_address`	Gauge 1, label `mac` da estação.
`esp_status_online`	Gauge 1 se STA conectado ao Wi-Fi, 0 caso contrário.
`esp_uptime_seconds`	Gauge, uptime em segundos pelo timer de 64 bits.
`esp_last_restart_reason`	Gauge 1, motivo no label `reason`.
`esp_boot_count`	Counter, inicializações persistidas desde início/reset dos contadores.
`esp_reboot_count`	Counter, `max(esp_boot_count - 1, 0)`, inclui ciclos de alimentação.
`esp_watchdog_resets`	Counter, resets identificados explicitamente como INT_WDT, TASK_WDT ou WDT.

Use `up{job="esp32c3mini"}` do próprio Prometheus para disponibilidade da coleta: uma placa desligada não publica seu estado. Um reset `panic` não permite inferir sozinho watchdog. O exporter não lê nem apaga core dumps.

Boots/watchdogs reutilizam `telemetry/boots`. `reset_counters` zera-os também em memória. Falha na persistência inicial deixa essas séries como `NaN`. Os outros contadores são voláteis e reiniciam com a placa.

Rede, CPU, tarefas e memória

Métrica	Unidade e escopo
`esp_wifi_rssi_dbm`	RSSI STA em dBm, NaN desconectado.
`esp_wifi_channel`	Canal STA, NaN desconectado.
`esp_wifi_disconnect_count`	Counter de eventos STA desconectado desde início do exporter, após a tentativa inicial de Wi-Fi.
`esp_wifi_tx_power`	Máximo TX configurado em dBm, convertido de unidades de 0,25 dBm; não medição instantânea.
`esp_cpu_frequency_mhz`	Frequência em MHz.
`esp_cpu_usage_percent`	Percentual agregado de CPU ocupado, calculado pelas estatísticas de runtime do FreeRTOS; precisa de duas amostras válidas separadas por pelo menos cinco segundos.
`esp_cpu_temperature_celsius`	Temperatura interna do chip em °C, pela mesma leitura disponível no comando `temp`; não é temperatura ambiente.
`esp_task_count`	Número atual de tarefas FreeRTOS.
`esp_free_rtos_tasks`	Alias da contagem de tarefas, não vagas livres.
`esp_heap_free_bytes`	Heap interno livre.
`esp_heap_total_bytes`	Heap interno total.
`esp_heap_minimum_free_bytes`	Mínimo de heap livre desde boot.
`esp_psram_free_bytes`	PSRAM livre; zero quando ausente.
`esp_psram_total_bytes`	PSRAM total; zero quando ausente.
`esp_flash_usage_percent`	Imagem do firmware / capacidade da partição de aplicação em execução × 100; não ocupação global da flash.

São gauges, exceto o contador de desconexões. Uso de CPU é `NaN` durante o aquecimento inicial, se o SDK não tiver estatísticas de runtime, se houver mais de 64 tarefas ou se não for possível medir todas as tarefas idle. O valor agrega os núcleos: considera o tempo ocioso total em relação à capacidade de processamento total no intervalo. No core Arduino ESP32 3.3.12 usado para validar este firmware, as estatísticas de runtime e o timer em microssegundos estão habilitados. A primeira leitura estabelece a referência; a métrica fica disponível após a amostra seguinte (cerca de 10 segundos depois do boot). Depois é atualizada a cada cinco segundos, independente do intervalo de coleta do Prometheus.

Ambiente e tela

Todas são gauges:

Métrica	Significado
`sensor_local_temperature_celsius{location="local",sensor="dht22"}`	Temperatura local em °C.

Métrica	Significado
`sensor_local_humidity_percent{location="local",sensor="dht22"}`	Umidade relativa local em %.
`sensor_temperature_celsius{source="open_meteo",location="porto_alegre"}`	Última temperatura externa disponível em °C (Porto Alegre).
`sensor_humidity_percent{source="open_meteo",location="porto_alegre"}`	Última umidade relativa externa disponível em % (Porto Alegre).
`sensor_pressure_hpa`	NaN, não há barômetro integrado.
`sensor_air_quality`	NaN, não há integração de qualidade do ar.
`esp_lcd_on`	1 ligada, 0 desligada.

O scrape usa a leitura compartilhada DHT22, respeitando os dois segundos; amostra inválida retorna NaN. O cache externo pode estar desatualizado após falhas e é NaN antes da primeira resposta válida. Não força consulta externa por scrape.

Integrações ainda indisponíveis

Os nomes abaixo, **exceto SD e logs**, são publicados como **gauges NaN**, mesmo os nomes que serão contadores quando forem integrados. SD e logs são omitidos da exportação. Não há montagem SD nem gravação de logs em cartão, GPIO monitorado, instrumentação elétrica ou relés no Mini atual. Isso inclui métricas de gravações/erros de log e uso do cartão usadas por alguns painéis Grafana: essas séries não são publicadas, para indicar No data.

Grupo	Métricas
Energia	`esp_voltage_v`, `esp_current_ma`, `esp_power_w`, `esp_battery_percent`, `esp_battery_voltage_v`
Relés	`relay_state`, `relay_activation_count`, `relay_runtime_seconds`

Grupo	Métricas
GPIO	`gpio_digital_input`, `gpio_digital_output`, `gpio_analog_value`
SD e logs	`sdcard_present`, `sdcard_mounted`, `sdcard_capacity_bytes`, `sdcard_total_bytes`, `sdcard_used_bytes`, `sdcard_free_bytes`, `sdcard_usage_percent`, `sdcard_read_errors_total`, `sdcard_write_errors_total`, `sdcard_log_size_bytes`, `sdcard_log_exists`, `sdcard_log_writes_total`, `sdcard_log_errors_total`, `sdcard_crash_reports_total`, `sdcard_crash_errors_total`, `sdcard_last_sample_uptime_seconds`

NaN não significa cartão ausente, relé desligado ou corrente zero: a medição não está implementada. São necessários hardware, pinos, escalas e parâmetros de conexão para torná-las reais. No dashboard, CPU interna deve apresentar dados após atualizar o firmware; painéis de SD e logs continuarão sem valores até que essas integrações sejam implementadas.

`esp_metric_supported{group="..."}` vale 0 para `energy`, `pressure`, `air_quality`, `relays`, `gpio` e `sdcard`, e 1 para `mqtt`, `dht22` e `cpu_temperature`. Para `cpu_usage`, vale 1 se o SDK tiver estatísticas de runtime habilitadas e 0 caso contrário. Indica suporte implementado, não presença física/validade da leitura.

Aplicação HTTP

Métrica	Tipo e escopo
`application_requests_total`	Counter de handlers concluídos na porta 80.
`application_errors_total`	Counter de respostas HTTP ≥ 400 nessa porta.
`application_response_time_ms`	Gauge, duração do último handler concluído, ms.
`application_response_time_ms_sum`	Counter, soma dessas durações, ms.
`application_response_time_ms_count`	Counter, quantidade de handlers concluídos.

Inclui dashboard, configurações, APIs e 404. Exclui listener de métricas, serial, UDP, falhas de sensores e comandos que retornam `[ERRO]` com HTTP 200. Mede entrada no handler até retorno, incluindo preparação/envio da resposta, sem espera anterior pela recepção do pedido. Um comando que reinicia antes de retornar não é contabilizado como concluído.

Validação e limites

- Compilação e gravação ficam com o usuário.
- Testar `/metrics`, coleta no Prometheus, troca de porta, persistência após reset e contadores após `reset_counters`.
- Testar portas inválidas: 80, 3232, zero, acima de 65535 e texto; testar intervalos abaixo de 5, acima de 3600 e texto.
- Dashboard continua na porta 80; a configuração salva não filtra clientes. O endpoint não possui autenticação e destina-se à rede local.
- Atendimento cooperativo no loop: consulta HTTPS de clima, sincronização e comandos longos podem atrasar o scrape.
- A integração Home Assistant requer PubSubClient >= 2.8, de Nick O'Leary.

Referências oficiais: formato Prometheus e coleta HTTP.